

PRÁCTICA N.º 7 — ENUNCIADO

Pruebas automatizadas de un microservicio

Pruebas unitarias, de integración con contenedor efímero y de contrato, listas para el pipeline

Asignatura	COM-600 · Microservicios · Gestión 2/2026
Tema del programa	Tema 5 — Desarrollo con Frameworks y Pruebas Automatizadas
Modalidad	Parte 1 guiada en laboratorio · Parte 2 individual
Duración estimada	Parte 1: 4 horas · Parte 2: trabajo autónomo
Puntaje	100 puntos (40 + 60)
Carpeta de trabajo	practica7-pruebas

Nota: Este documento es el ENUNCIADO: contiene los comandos, las explicaciones y los ejercicios. Las capturas y las respuestas NO se escriben aquí, sino en el documento complementario «Hoja de Evidencias», que es el que se entrega.

1. Objetivos

Al finalizar esta práctica el estudiante será capaz de:

- Distinguir prueba unitaria, de integración y de contrato, y saber cuál corresponde a cada situación.
- Escribir pruebas con la estructura preparar–ejecutar–verificar y nombres que describan la conducta esperada.
- Probar los casos borde y los errores, no solo el camino feliz.
- Medir la cobertura e interpretarla con criterio, sin convertirla en un fin en sí misma.
- Probar un endpoint HTTP completo sin levantar el servidor a mano.
- Ejecutar pruebas de integración contra una base de datos real levantada en un contenedor efímero.
- Escribir una prueba de contrato que detecte una ruptura entre dos microservicios.
- Dejar el proyecto en condiciones de que un pipeline de CI ejecute todo con un solo comando.

2. Qué necesita saber antes de empezar

El contenido conceptual de esta práctica se desarrolla en **Tema 5 — Desarrollo con Frameworks y Pruebas Automatizadas**. Aquí solo se recuerda lo mínimo necesario para ejecutar los laboratorios.

- **Prueba unitaria:** verifica una función o clase aislada, sin base de datos ni red. Se mide en milisegundos.
- **Prueba de integración:** verifica que dos o más piezas funcionen juntas: el endpoint con su repositorio, el repositorio con la base real.
- **Prueba de contrato:** verifica que el mensaje que un servicio produce siga siendo el que el otro espera consumir.
- **Contenedor efímero:** base de datos real que la propia suite levanta al empezar y destruye al terminar, para no depender de una instalación local.
- **Cobertura:** porcentaje de líneas o ramas que las pruebas ejecutaron. Dice qué se ejecutó, nunca si se verificó bien.

Repaso: La justificación conceptual —pirámide de pruebas, qué prueba cada nivel, costo de encontrar un error tarde— está en las diapositivas del Tema 5. Esta práctica es la ejecución.

3. Requisitos previos

Requisito	Verificación	Observación
Node.js 20 o superior	node --version	La versión 20 o la 22 LTS

Requisito	Verificación	Observación
npm 10+	npm --version	Viene con Node
Docker Desktop o Engine 24+	docker --version	Necesario para el Laboratorio 5
API de la Práctica 2	Carpeta practica2-docker	Se reutiliza la API de tareas; si no la tiene, el Laboratorio 1 la crea desde cero
Editor de código	VS Code recomendado	Extensión «Jest» de Orta para ver las pruebas en el editor
Git	git --version	Para subir la entrega al repositorio

Atención: Cree una carpeta de trabajo llamada practica7-pruebas y realice TODO el trabajo dentro de ella. Esa carpeta se sube al repositorio al final.

Los comandos se muestran precedidos por el símbolo \$. NO escriba el símbolo \$ al copiar el comando.

Ninguna prueba de esta práctica debe apuntar a una base de datos que usted use para otra cosa: las pruebas borran datos.

4. Estructura y forma de entrega

La práctica se apoya en dos documentos:

Documento	Para qué sirve	¿Se entrega?
Enunciado (este archivo)	Guía de trabajo: comandos, explicaciones, ejercicios y rúbrica. Se consulta mientras se trabaja.	No
Hoja de Evidencias	Formulario donde se pegan las 15 capturas y se responden las 7 preguntas de comprobación de la Parte 1.	Sí, en PDF

El trabajo tiene dos partes obligatorias:

Parte	Contenido	Puntaje
PARTE 1 — Guía	8 laboratorios paso a paso. Se ejecutan los comandos y se documenta cada resultado en la Hoja de Evidencias.	40 pts
PARTE 2 — Ejercicios propuestos	6 ejercicios de resolución individual. Se entregan como código en el repositorio más un informe en PDF.	60 pts
TOTAL		100 pts

Entregables

- Hoja de Evidencias en PDF, con las capturas pegadas y las preguntas de comprobación respondidas (Parte 1).
- Repositorio Git público con la carpeta practica7-pruebas: código del servicio, carpeta tests/, package.json con los scripts, jest.config.js y README.md con las instrucciones para correr la suite.
- Informe PDF de la Parte 2 con la resolución de cada ejercicio, capturas de la evidencia y las respuestas a las preguntas de reflexión.

PARTE 1 — PRÁCTICA GUIADA

Ejecute cada comando en el orden indicado. Cuando aparezca un aviso de «Captura N.º», tome la captura de pantalla y péguela en el recuadro correspondiente de la Hoja de Evidencias. Las «Pregunta N.º» se responden en esa misma hoja, con sus propias palabras: son las que demuestran que entendió lo que ejecutó.

Consejo: Trabaje con los dos documentos abiertos a la vez: este enunciado para leer y ejecutar, y la Hoja de Evidencias para ir registrando. Las capturas y las preguntas están numeradas igual en ambos.

Laboratorio 0 — Verificación del entorno

Antes de escribir una sola prueba, confirme que las herramientas están donde deben estar.

Paso 1. Verifique las versiones instaladas

Abra una terminal en la carpeta de trabajo y ejecute:

```
$ node --version
$ npm --version
$ docker --version
$ docker ps
```

El último comando debe responder con una tabla, aunque esté vacía. Si responde con un error de conexión, Docker no está iniciado.

Captura N.º 1: salida de los cuatro comandos anteriores en su terminal. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 1: ¿Por qué esta práctica necesita Docker si lo que vamos a escribir son pruebas y no contenedores de la aplicación? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 2. Cree el proyecto e instale el marco de pruebas

```
$ mkdir practica7-pruebas && cd practica7-pruebas
$ npm init -y
$ npm install express mongodb
$ npm install --save-dev jest supertest
# jest ejecuta las pruebas; supertest permite llamar a la API sin levantar el servidor
```

Captura N.º 2: salida de `npm install` mostrando los paquetes instalados y el archivo `package.json` creado. → *péguela en la Hoja de Evidencias.*

Paso 3. Declare el script de pruebas

Abra `package.json` y reemplace la sección "scripts" por la siguiente. El objetivo es que TODA la suite se ejecute con un solo comando: es lo que después hará el pipeline.

```
package.json (fragmento)
"scripts": {
  "test": "jest --runInBand",
  "test:watch": "jest --watch",
  "test:cov": "jest --coverage"
}
```

Laboratorio 1 — La primera prueba unitaria

Empezamos por la lógica de negocio pura: sin HTTP, sin base de datos. Es el nivel más barato y el más rápido de la pirámide.

Paso 1. Escriba la función que se va a probar

Cree la carpeta `src/` y dentro el archivo `tarea.js`. Es la regla de negocio del servicio de tareas: una tarea vencida y sin terminar es una tarea atrasada.

```
src/tarea.js
function estadoDeTarea(tarea, ahora = new Date()) {
  if (!tarea || !tarea.titulo) {
    throw new Error("La tarea necesita un título");
  }
  if (tarea.completada) return "COMPLETADA";
  if (tarea.vence && new Date(tarea.vence) < ahora) return "ATRASADA";
  return "PENDIENTE";
}

module.exports = { estadoDeTarea };
```

Detalle importante: La función recibe «ahora» como parámetro con un valor por omisión. Una función que lee el reloj por dentro es casi imposible de probar; una que lo recibe se prueba en un milisegundo.

Paso 2. Escriba la prueba con la estructura preparar–ejecutar–verificar

Cree la carpeta `tests/` y dentro el archivo `tarea.test.js`.

```
tests/tarea.test.js
const { estadoDeTarea } = require("../src/tarea");

describe("estadoDeTarea", () => {
  test("devuelve COMPLETADA cuando la tarea está terminada", () => {
    const tarea = { titulo: "Entregar informe", completada: true }; // preparar
    const estado = estadoDeTarea(tarea); // ejecutar
    expect(estado).toBe("COMPLETADA"); // verificar
  });
});
```

Paso 3. Ejecute la suite

```
$ npm test
```

Debe ver una prueba en verde con el tiempo que tardó. Si aparece en rojo, lea el mensaje: Jest muestra qué esperaba y qué recibió.

Captura N.º 3: salida de `npm test` con la primera prueba pasando en verde. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 2: El nombre de la prueba es «devuelve COMPLETADA cuando la tarea está terminada» y no «test1». ¿Qué gana el equipo cuando esa prueba falla dentro de seis meses? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 2 — Casos borde y errores: donde viven los defectos

El camino feliz casi nunca es el que rompe en producción. Ahora probamos los límites.

Paso 1. Agregue los casos que faltan

Añada estas pruebas al mismo archivo, dentro del describe.

```
tests/tarea.test.js (agregar)
```

```
test("devuelve ATRASADA cuando venció y no está completada", () => {
  const tarea = { titulo: "Pagar servidor", vence: "2026-01-10" };
  expect(estadoDeTarea(tarea, new Date("2026-01-11"))).toBe("ATRASADA");
});

test("una tarea completada nunca figura como atrasada", () => {
  const tarea = { titulo: "Pagar servidor", vence: "2026-01-10", completada: true };
  expect(estadoDeTarea(tarea, new Date("2026-06-01"))).toBe("COMPLETADA");
});

test("lanza un error si la tarea no tiene título", () => {
  expect(() => estadoDeTarea({})).toThrow("La tarea necesita un título");
});
```

Captura N.º 4: salida de npm test con las cuatro pruebas del bloque en verde. → *péguela en la Hoja de Evidencias.*

Paso 2. Provoque una falla a propósito

En src/tarea.js cambie el operador « < » por « <= » en la comparación de la fecha y vuelva a ejecutar la suite. Observe el mensaje de error completo antes de revertir el cambio.

```
$ npm test
```

Captura N.º 5: salida de la prueba en rojo, mostrando el valor esperado y el recibido. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 3: Una prueba que nunca ha fallado no demuestra nada. ¿Cómo puede comprobar que una prueba realmente verifica lo que dice verificar? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 3 — Cobertura: qué mide y qué no

Paso 1. Genere el reporte de cobertura

```
$ npm run test:cov
```

Jest imprime una tabla con el porcentaje de líneas, ramas, funciones y sentencias, y deja un reporte navegable en coverage/lcov-report/index.html.

Captura N.º 6: tabla de cobertura impresa en la terminal. → *péguela en la Hoja de Evidencias.*

Paso 2. Abra el reporte y busque el rojo

Abra coverage/lcov-report/index.html en el navegador y entre a src/tarea.js. Las líneas en rojo no fueron ejecutadas por ninguna prueba.

Captura N.º 7: reporte HTML de cobertura mostrando el archivo src/tarea.js con sus líneas coloreadas. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 4: Escriba una prueba que ejecute todas las líneas de estadoDeTarea pero que no verifique nada útil (por ejemplo, sin ningún expect que compare el resultado). ¿Qué cobertura reporta? ¿Qué conclusión saca sobre la cobertura como criterio de calidad? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 3. Fije un umbral mínimo

Cree jest.config.js en la raíz del proyecto. El umbral no persigue un número: impide que la cobertura baje sin que nadie se entere.

```
jest.config.js
module.exports = {
  testEnvironment: "node",
  coverageThreshold: {
    global: { branches: 70, functions: 80, lines: 80, statements: 80 }
  }
};
```

Laboratorio 4 — Probar el endpoint HTTP sin levantar el servidor

Subimos un nivel de la pirámide: ahora se prueba la ruta completa, con su enrutamiento, su validación y sus códigos de estado.

Paso 1. Separe la aplicación del servidor

Este es el cambio que hace probable una API. app.js define la aplicación; server.js es el único que la pone a escuchar. Las pruebas importan app.js y nunca abren un puerto.

```
src/app.js
const express = require("express");
const { estadoDeTarea } = require("./tarea");

const app = express();
app.use(express.json());
const tareas = [];

app.post("/tareas", (req, res) => {
  if (!req.body.titulo) return res.status(400).json({ error: "titulo es obligatorio" });
  const tarea = { id: tareas.length + 1, ...req.body, completada: false };
  tareas.push(tarea);
  res.status(201).json(tarea);
});

app.get("/tareas", (_req, res) => res.json(
  tareas.map(t => ({ ...t, estado: estadoDeTarea(t) }))
));

module.exports = app;
```

Paso 2. Escriba la prueba de integración HTTP

```
tests/api.test.js
const request = require("supertest");
const app = require("../src/app");

test("POST /tareas crea la tarea y responde 201", async () => {
  const res = await request(app).post("/tareas").send({ titulo: "Estudiar" });
  expect(res.status).toBe(201);
  expect(res.body).toHaveProperty("id");
});
```

```
test("POST /tarefas sin título responde 400 y no crea nada", async () => {
  const res = await request(app).post("/tarefas").send({});
  expect(res.status).toBe(400);
  expect(res.body.error).toMatch(/título/);
});
```

Captura N.º 8: salida de npm test con las pruebas de la API en verde. → *péguela en la Hoja de Evidencias.*

Paso 3. Compruebe que sigue siendo rápido

Ejecute la suite y observe el tiempo total. Todavía no hay base de datos: estas pruebas deben seguir midiéndose en milisegundos.

```
$ npm test
```

Captura N.º 9: salida completa de la suite con el tiempo total de ejecución visible. → *péguela en la Hoja de Evidencias.*

Laboratorio 5 — Integración real con un contenedor efímero

Una prueba contra una base simulada demuestra poco. Aquí la propia suite levanta MongoDB en un contenedor, corre contra él y lo destruye al terminar.

Paso 1. Instale Testcontainers

```
$ npm install --save-dev @testcontainers/mongodb testcontainers
# la primera ejecución descarga la imagen de MongoDB: puede tardar unos minutos
```

Paso 2. Escriba el repositorio que habla con la base

```
src/repositorio.js
const { MongoClient } = require("mongodb");

async function crearRepositorio(uri) {
  const cliente = await new MongoClient(uri).connect();
  const col = cliente.db("practica7").collection("tarefas");
  return {
    guardar: (t) => col.insertOne(t),
    buscarPorTitulo: (titulo) => col.findOne({ titulo }),
    contar: () => col.countDocuments(),
    cerrar: () => cliente.close(),
  };
}

module.exports = { crearRepositorio };
```

Paso 3. Escriba la prueba de integración con la base real

```
tests/repositorio.int.test.js
const { MongoDBContainer } = require("@testcontainers/mongodb");
const { crearRepositorio } = require("../src/repositorio");

let contenedor, repo;

beforeAll(async () => {
  contenedor = await new MongoDBContainer("mongo:7").start();
  repo = await crearRepositorio(contenedor.getConnectionString() +
    "?directConnection=true");
});
```

```
}, 120000);

afterAll(async () => { await repo.cerrar(); await contenedor.stop(); });

test("guarda una tarea y la recupera por título", async () => {
  await repo.guardar({ titulo: "Leer Newman", completada: false });
  const encontrada = await repo.buscarPorTitulo("Leer Newman");
  expect(encontrada.completada).toBe(false);
});
```

Por qué 120000: El tiempo de espera de beforeAll se amplía porque la primera vez hay que descargar la imagen. Sin ese ajuste, la prueba falla por tiempo agotado y parece un error del código.

Paso 4. Ejecute y observe el contenedor mientras corre

Abra una segunda terminal y ejecute docker ps mientras la suite está corriendo. Verá el contenedor de MongoDB con un nombre generado y un puerto aleatorio.

```
# terminal 1
$ npm test

# terminal 2, mientras la suite corre
$ docker ps
```

Captura N.º 10: las dos terminales: la suite ejecutándose y docker ps mostrando el contenedor efímero levantado por la prueba. → *péguela en la Hoja de Evidencias*.

Paso 5. Confirme que no quedó nada

Cuando la suite termina, el contenedor debe desaparecer solo.

```
$ docker ps -a | grep mongo
```

Captura N.º 11: salida mostrando que el contenedor de la prueba ya no existe después de terminar la suite. → *péguela en la Hoja de Evidencias*.

Pregunta N.º 5: ¿Qué problema concreto evita este enfoque frente a la alternativa de que cada estudiante instale MongoDB en su propia máquina y las pruebas apunten a esa instalación? → *respóndala en la Hoja de Evidencias, con sus propias palabras*.

Laboratorio 6 — Prueba de contrato entre dos servicios

El servicio de Notificaciones consume los eventos que publica el de Tareas. Si Tareas renombra un campo, las pruebas unitarias de ambos siguen en verde y la integración se rompe en producción. La prueba de contrato existe para eso.

Paso 1. Declare el contrato como archivo versionado

El contrato es un artefacto compartido: vive en el repositorio y cambia con revisión, no por descuido.

```
contratos/tarea-creada.contrato.json
{
  "evento": "TareaCreada",
  "version": "1.0",
  "camposObligatorios": ["id", "titulo", "creadaEn"],
  "tipos": { "id": "number", "titulo": "string", "creadaEn": "string" }
```

```
}
```

Paso 2. Escriba la prueba del lado del proveedor

```
tests/contrato.test.js
const contrato = require("../contratos/tarea-creada.contrato.json");
const { construirEventoTareaCreada } = require("../src/eventos");

test("el evento publicado cumple el contrato acordado", () => {
  const evento = construirEventoTareaCreada({ id: 1, titulo: "Estudiar" });
  for (const campo of contrato.camposObligatorios) {
    expect(evento).toHaveProperty(campo);
    expect(typeof evento[campo]).toBe(contrato.tipos[campo]);
  }
});
```

Cree src/eventos.js con la función construirEventoTareaCreada que arme el objeto con los tres campos del contrato y ejecute la suite.

Captura N.º 12: la prueba de contrato en verde junto al resto de la suite. → *péguela en la Hoja de Evidencias.*

Paso 3. Rompa el contrato a propósito

En src/eventos.js renombre el campo «creadaEn» a «fecha», como haría un desarrollador que no sabe quién consume el evento, y ejecute la suite.

```
$ npm test
```

Captura N.º 13: la prueba de contrato fallando y señalando el campo faltante. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 6: Las pruebas unitarias del servicio de Tareas siguieron en verde con ese cambio. ¿Por qué? ¿Qué tipo de defecto detecta la prueba de contrato que ningún otro nivel detecta? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Laboratorio 7 — Dejar el proyecto listo para el pipeline

La práctica termina donde empieza la siguiente: todo esto debe poder ejecutarlo una máquina, sin usted delante.

Paso 1. Separe las suites por velocidad

Las pruebas rápidas deben poder correr solas y en cada guardado; las lentas, cuando corresponda. Agregue estos scripts al package.json.

```
package.json (fragmento)
"scripts": {
  "test": "jest --runInBand",
  "test:unit": "jest --testPathIgnorePatterns int.test",
  "test:int": "jest int.test --runInBand",
  "test:cov": "jest --coverage"
}
```

Paso 2. Verifique el criterio de salida

Un pipeline decide con el código de salida del proceso: 0 si todo pasó, distinto de 0 si algo falló. Compruébelo.

```
$ npm test
$ echo $?          # en Linux o macOS
$ echo $LASTEXITCODE # en PowerShell
```

Rompa una prueba, vuelva a ejecutar y compare el valor. Ese número es toda la conversación entre sus pruebas y el pipeline.

Captura N.º 14: los dos códigos de salida: 0 con la suite en verde y distinto de 0 con la suite en rojo. → *péguela en la Hoja de Evidencias.*

Pregunta N.º 7: Si una prueba es inestable (a veces pasa y a veces falla), ¿qué le ocurre al pipeline y qué le ocurre a la confianza del equipo? ¿Qué haría usted con esa prueba? → *respóndala en la Hoja de Evidencias, con sus propias palabras.*

Paso 3. Documente cómo se corre todo

Escriba un README.md que indique: requisitos, cómo instalar, cómo correr cada suite y cuánto tarda cada una. Súbalo junto con el resto al repositorio.

```
$ git init && git add .
$ git commit -m "Práctica 7: suite de pruebas automatizadas"
$ git remote add origin <URL-de-su-repositorio>
$ git push -u origin main
```

Captura N.º 15: la página del repositorio en GitHub mostrando la carpeta practica7-pruebas con tests/, contratos/ y el README. → *péguela en la Hoja de Evidencias.*

PARTE 2 — EJERCICIOS PROPUESTOS

Esta parte se resuelve de forma individual. Cada ejercicio se entrega con: (a) el código o los archivos de configuración en el repositorio, (b) las capturas que demuestren su funcionamiento y (c) la explicación pedida, en el informe PDF.

Nota: Los ejercicios están ordenados por dificultad creciente. Del 1 al 4 son obligatorios para todos; el 5 y el 6 son los que separan una entrega correcta de una sobresaliente.

Ejercicio 1. Suite unitaria de una regla de negocio propia (10 pts)

Elija una regla de negocio de su proyecto integrador (cálculo de un total con descuento, validación de un documento de identidad, cálculo de mora por días de retraso, asignación de cupo) e impleméntela como función pura. Escriba la suite que la cubra.

Se pide:

- Al menos seis pruebas: el camino feliz, tres casos borde y dos casos de error.
- Nombres de prueba que describan la conducta esperada, no el número de caso.
- La estructura preparar–ejecutar–verificar visible en cada prueba.
- Ninguna dependencia de reloj, red o base de datos: si la regla necesita la fecha, se recibe como parámetro.

Criterio de aceptación: *Un compañero debe poder leer solo los nombres de las pruebas y entender qué hace la función.*

Ejercicio 2. Primero la prueba que falla (10 pts)

Se le entrega esta función con un defecto real: al aplicar el descuento no controla el límite y puede devolver un total negativo.

```
src/total.js (con defecto)
function calcularTotal(items, descuentoPorcentaje) {
  const bruto = items.reduce((a, i) => a + i.precio * i.cantidad, 0);
  return bruto - (bruto * descuentoPorcentaje / 100);
}
```

Se pide:

- Escriba primero una prueba que falle y que demuestre el defecto (por ejemplo, un descuento de 150).
- Ejecute la suite y capture la prueba en rojo.
- Corrija la función y capture la misma prueba en verde, sin modificar la prueba.
- Explique en el informe por qué el orden importa: prueba en rojo, después corrección.

Criterio de aceptación: *La captura en rojo debe existir. Una corrección sin evidencia previa de la falla no acredita el ejercicio.*

Ejercicio 3. Integración con base de datos efímera (12 pts)

Amplíe el repositorio del Laboratorio 5 con dos operaciones nuevas: actualizar el estado de una tarea y eliminarla. Pruébelas contra el contenedor efímero.

Se pide:

- Al menos cuatro pruebas de integración que cubran las cuatro operaciones del repositorio.
- Una prueba que verifique el comportamiento cuando el documento no existe.
- Aislamiento entre pruebas: cada prueba debe poder correr sola y en cualquier orden (limpie la colección donde corresponda).
- Captura de docker ps mostrando el contenedor durante la ejecución.

Criterio de aceptación: *Ejecutar la suite dos veces seguidas debe dar el mismo resultado sin intervención manual.*

Ejercicio 4. Contrato entre dos de sus servicios (10 pts)

Tome dos servicios de su proyecto integrador que se comuniquen (por API o por evento) y formalice el contrato entre ambos.

Se pide:

- El contrato como archivo versionado en la carpeta contratos/.
- Una prueba del lado del proveedor que verifique que el mensaje cumple el contrato.
- Una prueba del lado del consumidor que verifique que sabe procesar ese mensaje.
- Una demostración de ruptura: cambie el proveedor y muestre cuál prueba lo detecta.

Criterio de aceptación: *La ruptura debe ser detectada por la prueba de contrato, no por una prueba de extremo a extremo.*

Ejercicio 5. Cobertura con criterio (8 pts)

Mida la cobertura de su proyecto, súbala de forma significativa y argumente la decisión.

Se pide:

- Reporte de cobertura antes y después, con los porcentajes de líneas y de ramas.
- Identificación de tres zonas no cubiertas y justificación de cuáles vale la pena cubrir y cuáles no.
- Un umbral configurado en jest.config.js y la evidencia de que la suite falla si la cobertura baja de él.

Criterio de aceptación: Se evalúa el criterio, no el porcentaje: una justificación honesta de por qué NO cubrir algo puntúa igual que cubrirlo.

Ejercicio 6. Proyecto listo para integración continua (10 pts)

Deje su proyecto en condiciones de que una máquina, en un entorno limpio y sin usted delante, pueda verificarlo por completo.

Se pide:

- Un único comando que instale, ejecute análisis estático y corra todas las pruebas.
- Suites separadas por velocidad (rápidas y lentas) y documentadas en el README con sus tiempos.
- Evidencia del código de salida en verde y en rojo.
- Una lista de al menos tres condiciones que, según usted, deberían impedir que un cambio se fusione a main.

Criterio de aceptación: Se probará clonando su repositorio en una máquina limpia y ejecutando el comando que documente. Si necesita un paso manual no documentado, el ejercicio no acredita.

Rúbrica de evaluación

Parte 1 — Práctica guiada (40 puntos)

Criterio	Descripción	Pts
Capturas completas	Las 15 capturas están pegadas en la Hoja de Evidencias, son legibles y corresponden a la máquina del estudiante (se ve su usuario o el nombre del equipo).	15
Ejecución correcta	Las salidas muestran el resultado esperado, sin errores sin resolver.	10
Preguntas de comprobación	Las 7 preguntas están respondidas con criterio propio, no copiadas del enunciado.	10
Orden y presentación	Hoja de Evidencias completa, con datos del estudiante y declaración firmada, entregada en PDF y dentro del plazo.	5
Subtotal		40

Parte 2 — Ejercicios propuestos (60 puntos)

Ejercicio	Qué se evalúa	Pts
1. Suite unitaria de una regla de negocio propia	Seis pruebas con casos borde y de error, nombres descriptivos, función pura sin dependencias ocultas.	10
2. Primero la prueba que falla	Prueba que falla antes de la corrección, corrección mínima, evidencia de ambos estados y explicación del orden.	10
3. Integración con base de datos efímera	Cuatro operaciones probadas, caso de documento inexistente, aislamiento real entre pruebas, evidencia del contenedor.	12
4. Contrato entre dos de sus servicios	Contrato versionado, pruebas de ambos lados, demostración de la ruptura detectada por el nivel correcto.	10
5. Cobertura con criterio	Medición antes y después, análisis de las zonas descubiertas, umbral configurado y demostrado.	8
6. Proyecto listo para integración continua	Comando único funcional en máquina limpia, suites separadas y documentadas, códigos de salida, criterios de fusión razonados.	10
Subtotal		60

Penalizaciones

Situación	Descuento
Pruebas que dependen del orden de ejecución o de datos dejados por otra prueba	-5 pts

Situación	Descuento
Pruebas sin ninguna verificación (sin expect) usadas para inflar la cobertura	-5 pts
node_modules o coverage subidos al repositorio (falta .gitignore)	-3 pts
Pruebas que apuntan a una base de datos instalada localmente en lugar del contenedor efímero	-3 pts
Capturas que no corresponden a la máquina del estudiante	Anulación de la Parte 1
Entrega fuera de plazo	Según reglamento de la materia

Anexo — Errores frecuentes y cómo resolverlos

Mensaje o síntoma	Causa probable	Solución
Cannot find module "../src/tarea"	La ruta relativa parte del archivo de prueba, no de la raíz del proyecto.	Verifique la estructura de carpetas: tests/ y src/ deben ser hermanas.
Timeout - Async callback was not invoked within 5000 ms	La descarga de la imagen o el arranque del contenedor tarda más que el tiempo de espera por omisión.	Pase el tercer argumento a beforeAll (por ejemplo 120000) y precargue la imagen con docker pull mongo:7.
Could not find a working container runtime	Docker no está iniciado o el usuario no tiene permisos sobre el socket.	Inicie Docker Desktop y confirme con docker ps antes de correr la suite.
MongoServerSelectionError	La cadena de conexión del contenedor necesita directConnection=true.	Agregue ese parámetro a la URI que devuelve getConnectionString().
Jest did not exit one second after the test run	Quedó una conexión abierta: el cliente de Mongo o el servidor HTTP no se cerraron.	Cierre todo en afterAll; use --detectOpenHandles para ubicar qué quedó abierto.
Las pruebas pasan solas pero fallan juntas	Comparten estado: la misma colección, el mismo arreglo en memoria.	Limpie el estado en beforeEach y evite datos globales entre pruebas.
La cobertura sale en 0 %	Jest no encuentra el código fuente porque solo se importó dentro de un mock.	Revise collectCoverageFrom en jest.config.js y que las pruebas importen el módulo real.

Antes de pedir ayuda: Antes de pedir ayuda, ejecute la suite con `npx jest --verbose` y lea el nombre de la prueba que falla, el valor esperado y el recibido. Esos tres datos resuelven la mayoría de los casos sin ayuda de nadie.

Referencias

- Documentación oficial de Jest — <https://jestjs.io/docs/getting-started>
- Supertest — <https://github.com/ladjs/supertest>
- Testcontainers para Node.js — <https://node.testcontainers.org/>
- Richardson, Chris. Microservices Patterns, capítulo sobre estrategias de prueba. Manning, 2018.
- Diapositivas del Tema 5 — Desarrollo con Frameworks y Pruebas Automatizadas (COM-600).